ASSIGNMENT 1: PAYMENT CALCULATOR FOR A VILLAGE MILK COOPERATIVE CSA0809– PYTHON PROGRAMMING

Submitted By:

- **Name: Y. Siva Pardhav**
- **Register Number: 192412612**
- **Course / Section: CSA0809**

Repository Link: https://github.com/Pardhav2006/Python

Q1 – Decisions with Many Boundaries

1. Problem Context & Boundary Predictions

A household is eligible for an electricity subsidy based on consumption slabs: <=100 units (full subsidy), 101-200 units (partial subsidy only if holding a ration card), >200 units (no subsidy, unless a member is >80 years of age).

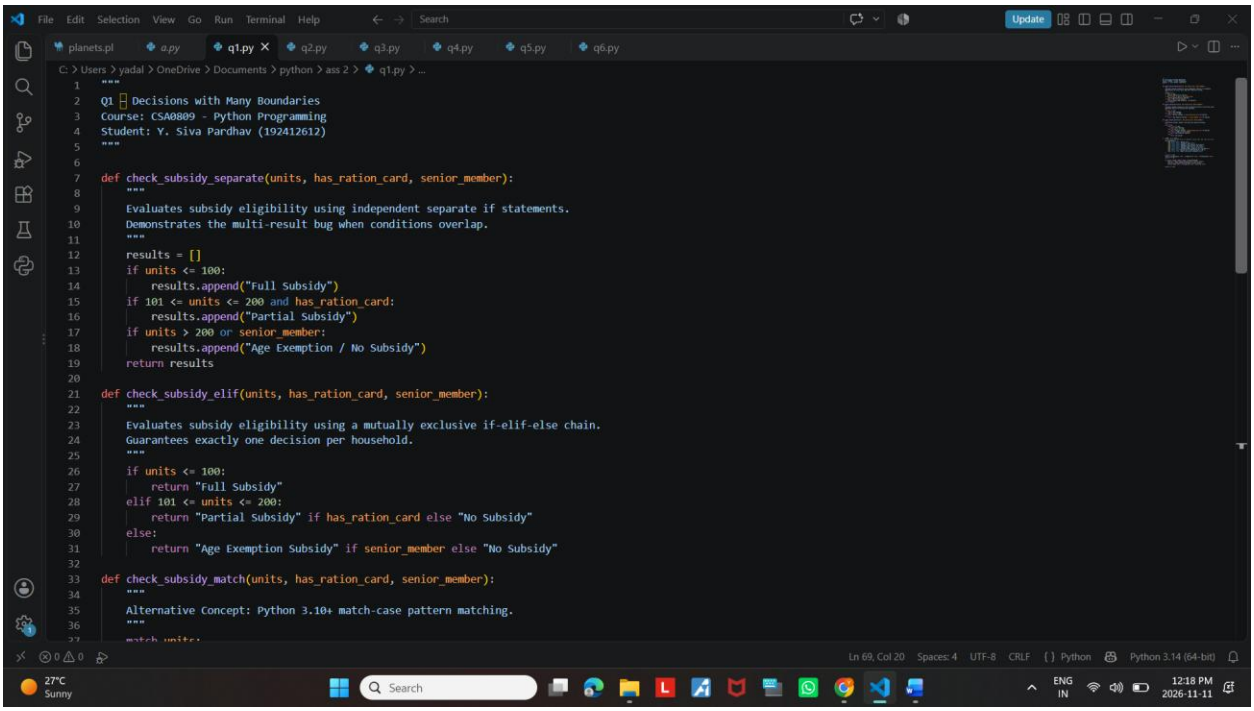
Household Case	Predicted Decision	Boundary Justification
50 units	Full Subsidy	Consumption <= 100 units.
100 units	Full Subsidy	Lower boundary threshold for full subsidy.
101 units (With Card)	Partial Subsidy	Lower boundary condition requiring ration card.
101 units (No Card)	No Subsidy	Falls into partial slab without card.
200 units	Partial Subsidy	Upper boundary for partial subsidy slab.
201 units (No Senior)	No Subsidy	Exceeds 200 without age exemption.
90 units (Senior >80)	Full Subsidy / Overlap	Matches <= 100 rule first.

2. Python Implementation

```
def check_subsidy_separate(units, has_ration_card, senior_member):
    """
    Evaluates subsidy eligibility using independent separate if statements.
    Demonstrates the multi-result bug when conditions overlap.
    """
    results = []
    if units <= 100:
        results.append("Full Subsidy")
    if 101 <= units <= 200 and has_ration_card:
        results.append("Partial Subsidy")
    if units > 200 or senior_member:
        results.append("Age Exemption / No Subsidy")
    return results
def check_subsidy_elif(units, has_ration_card, senior_member):
    """
    Evaluates subsidy eligibility using a mutually exclusive if-elif-else chain.
    Guarantees exactly one decision per household.
    """
```

```
    if units <= 100:
        return "Full Subsidy"
    elif 101 <= units <= 200:
        return "Partial Subsidy" if has_ration_card else "No Subsidy"
    else:
        return "Age Exemption Subsidy" if senior_member else "No Subsidy"
def check_subsidy_match(units, has_ration_card, senior_member):
    """
    Alternative Concept: Python 3.10+ match-case pattern matching.
    """
    match units:
        case u if u <= 100:
            return "Full Subsidy"
        case u if 101 <= u <= 200:
            return "Partial Subsidy" if has_ration_card else "No Subsidy"
        case u if u > 200 and senior_member:
            return "Age Exemption Subsidy"
        case _:
            return "No Subsidy"
```

```
if __name__ == "__main__":
    # Test cases covering normal and boundary values (100, 101, 200, 201 units)
    test_households = [
        (50, False, False, "Normal Low"),
        (100, False, False, "Boundary Lower Full"),
        (101, True, False, "Boundary Lower Partial (with Card)"),
        (101, False, False, "Boundary Lower Partial (no Card)"),
        (200, True, False, "Boundary Upper Partial (with Card)"),
        (201, False, False, "Boundary Upper Exemption Test (no Senior)"),
        (90, False, True, "Overlap Test (<=100 and Senior > 80)"),
        (250, False, True, "Upper Slab Senior Exemption Test")
    print("=" * 80)
    print(f"{'Description':<38} | {'Separate IF':<28} | {'IF-ELIF-ELSE':<20}")
    print("=" * 80)
    for units, ration, senior, desc in test_households:
        sep_res = check_subsidy_separate(units, ration, senior)
        elif_res = check_subsidy_elif(units, ration, senior)
        print(f"{desc:<38} | {str(sep_res):<28} | {elif_res:<20}")
    print("=" * 80)
```

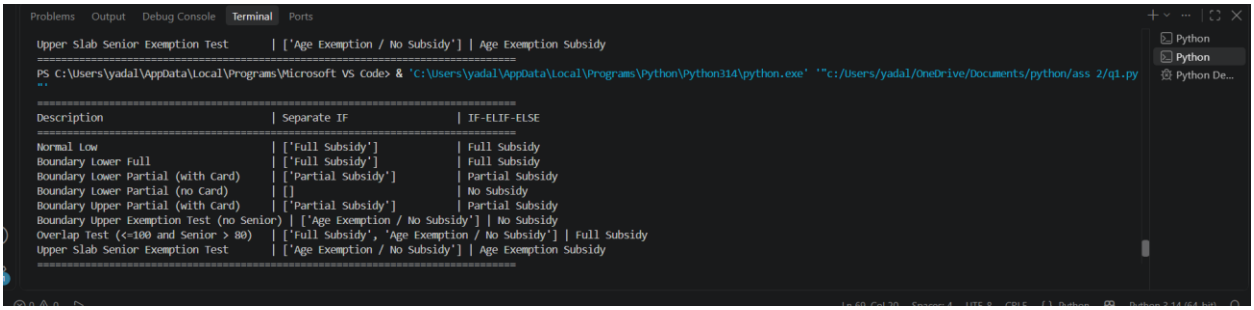


```
"""
Q1 – Decisions with Many Boundaries
Course: CSA0809 - Python Programming
Student: Y. Siva Pardhav (192412612)
"""

def check_subsidy_separate(units, has_ration_card, senior_member):
    """
    Evaluates subsidy eligibility using independent separate if statements.
    Demonstrates the multi-result bug when conditions overlap.
    """
    results = []
    if units <= 100:
        results.append("Full Subsidy")
    if 101 <= units <= 200 and has_ration_card:
        results.append("Partial Subsidy")
    if units > 200 or senior_member:
        results.append("Age Exemption / No Subsidy")
    return results

def check_subsidy_elif(units, has_ration_card, senior_member):
    """
    Evaluates subsidy eligibility using a mutually exclusive if-elif-else chain.
    Guarantees exactly one decision per household.
    """
    if units <= 100:
        return "Full Subsidy"
    elif 101 <= units <= 200:
        return "Partial Subsidy" if has_ration_card else "No Subsidy"
    else:
        return "Age Exemption Subsidy" if senior_member else "No Subsidy"

def check_subsidy_match(units, has_ration_card, senior_member):
    """
    Alternative Concept: Python 3.10+ match-case pattern matching.
    """
    match units:
```



```
Upper Slab Senior Exemption Test       | ['Age Exemption / No Subsidy'] | Age Exemption Subsidy
================================================================================
PS C:\Users\yadal\AppData\Local\Programs\Microsoft VS Code> & 'C:\Users\yadal\AppData\Local\Programs\Python\Python314\python.exe' '"c:/Users/yadal/OneDrive/Documents/python/ass 2/q1.py
"'

================================================================================
Description                            | Separate IF                    | IF-ELIF-ELSE
================================================================================
Normal Low                             | ['Full Subsidy']               | Full Subsidy
Boundary Lower Full                    | ['Full Subsidy']               | Full Subsidy
Boundary Lower Partial (with Card)     | ['Partial Subsidy']            | Partial Subsidy
Boundary Lower Partial (no Card)       | []                             | No Subsidy
Boundary Upper Partial (with Card)     | ['Partial Subsidy']            | Partial Subsidy
Boundary Upper Exemption Test (no Senior) | ['Age Exemption / No Subsidy'] | No Subsidy
Overlap Test (<=100 and Senior > 80)   | ['Full Subsidy', 'Age Exemption / No Subsidy'] | Full Subsidy
Upper Slab Senior Exemption Test       | ['Age Exemption / No Subsidy'] | Age Exemption Subsidy
================================================================================
```

Q2 – Loops that Count Attempts

1. Problem Context: OTP Verification

A bank kiosk allows a customer 3 attempts to enter an OTP. After 3 wrong attempts, the account locks for 30 minutes. Blank entries should not count as attempts.

"""

Q2 – Loops that Count Attempts (OTP Verification at a Bank Kiosk)

Course: CSA0809 - Python Programming

Student: Y. Siva Pardhav (192412612)

"""

```
def verify_otp_while(correct_otp="1234"):
    """
    Simulates OTP verification using a while loop.
    Blank entries do not consume an attempt.
    """
    print("\n--- Starting OTP Verification (WHILE Loop) ---")
    attempts = 0
    max_attempts = 3

    while attempts < max_attempts:
        otp = input(f"Attempt {attempts + 1}/3 - Enter OTP (or leave blank to skip): ").strip()

        # Blank entry handling
        if otp == "":
            print("→ Blank entry detected. This does not count as an attempt.")
            continue  # Skips incrementing attempts and loops back

        if otp == correct_otp:
            print("→ Access Granted! Welcome to your account.")
            break
        else:
            print("→ Incorrect OTP.")
            attempts += 1
    else:
        # Executed only if the loop finishes without hitting 'break'
```

```
        print("→ ACCOUNT LOCKED for 30 minutes due to 3 failed attempts.")


def verify_otp_for(correct_otp="1234"):
    """
    Simulates OTP verification using a for loop with range(3).
    Demonstrates the use of pass and manual attempt tracking.
    """
    print("\n--- Starting OTP Verification (FOR Loop) ---")
    max_attempts = 3
    failed_attempts = 0

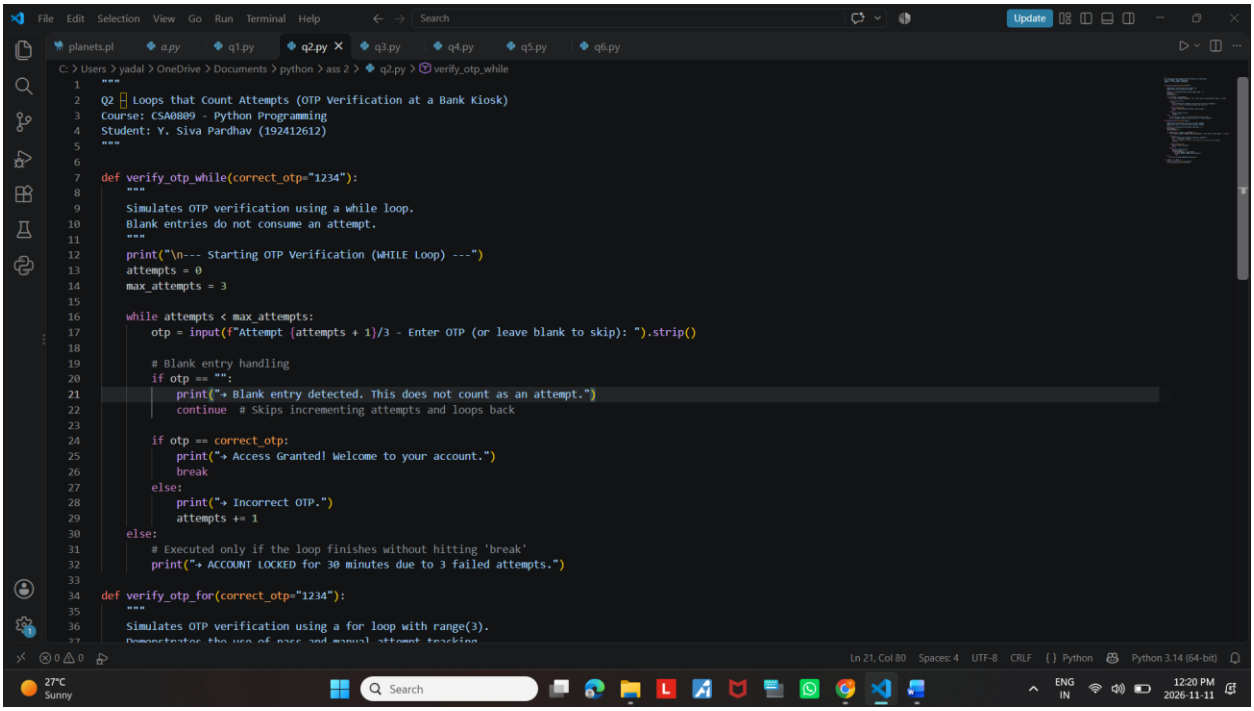
    for attempt_num in range(1, max_attempts + 1):
        otp = input(f"Attempt {attempt_num}/{max_attempts} - Enter OTP (or leave blank): ").strip()

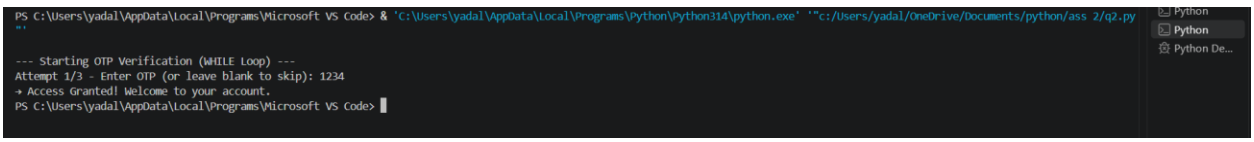
        if otp == "":
            print("→ Blank entry detected. Using pass statement.")
            pass  # Placeholder statement
            # Note: A standard for loop in range consumes an iteration unless managed
            continue

        if otp == correct_otp:
            print("→ Access Granted!")
            break
        else:
            print("→ Incorrect OTP.")
            failed_attempts += 1
```

```
        if failed_attempts >= max_attempts:
            print("→ ACCOUNT LOCKED for 30 minutes.")
            break
    else:
        print("→ For loop completed successfully.")


if __name__ == "__main__":
    # Run the while loop verification demo
    verify_otp_while(correct_otp="1234")
```





Q3 – Vehicle Registration Number Validator

1. Implementation Using String Methods and Slicing

Valid vehicle format: 2 letters (state) + 2 digits (district) + 1 or 2 letters (series) + 4 digits. Implemented using slicing, isalpha(), and isdigit() without regular expressions.

```
"""

Q3 – Engineer a Vehicle Registration Number Validator

Course: CSA0809 - Python Programming

Student: Y. Siva Pardhav (192412612)

"""


def normalised(text):

    """'tn 09 ab 1234' -> 'TN09AB1234' (spaces and hyphens removed, upper case)."""

    # TODO 1: Remove spaces and hyphens and convert to upper case using string methods

    return text.replace(" ", "").replace("-", "").upper()


def is_valid(reg):

    """Valid form: 2 letters, 2 digits, 1 or 2 letters, 4 digits (e.g. TN09AB1234)."""

    # TODO 2: Check the length is 9 or 10, then check each part using slicing with isalpha() and isdigit()

    if len(reg) == 9:

        # Format: LL DD L DDDD (Length 9)

        return (reg[0:2].isalpha() and

                reg[2:4].isdigit() and

                reg[4:5].isalpha() and

                reg[5:9].isdigit())
```

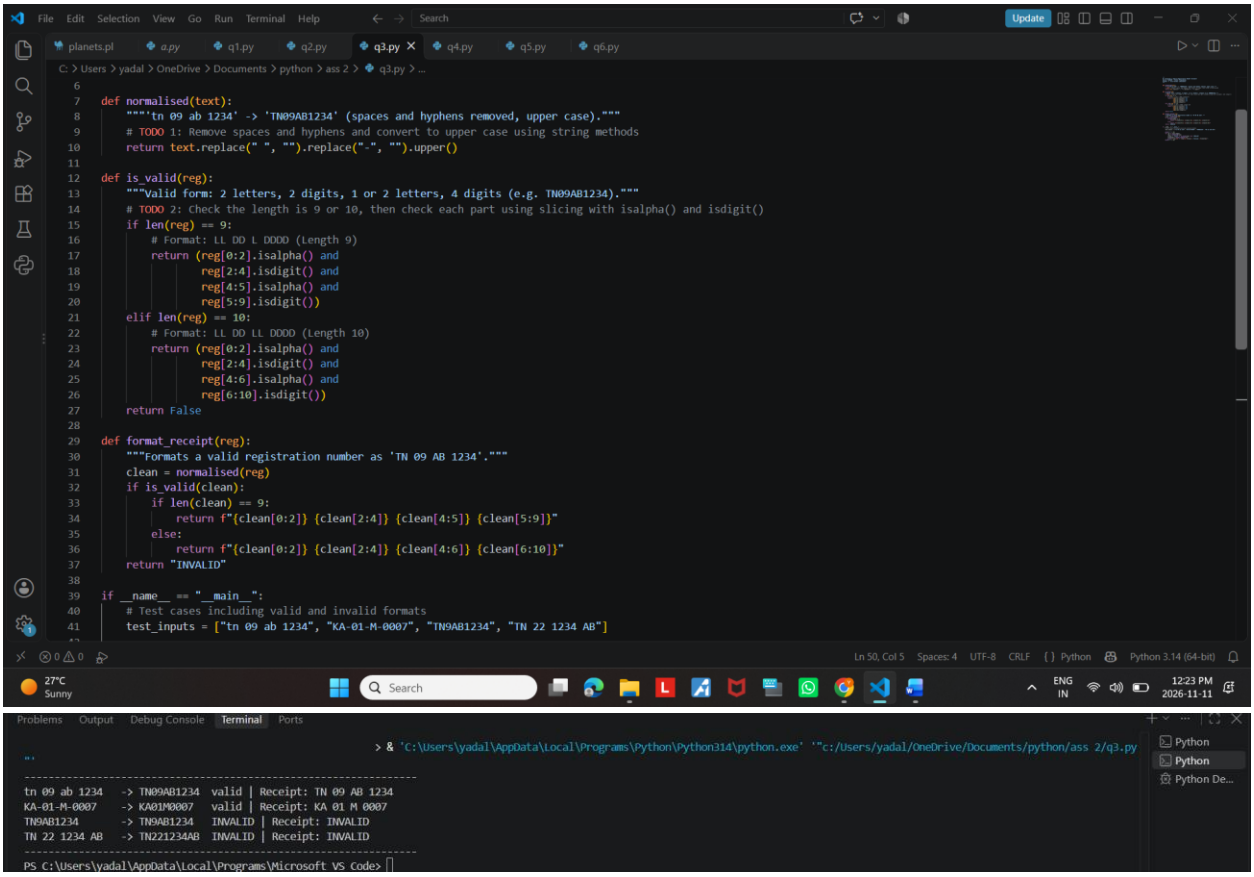
```
    elif len(reg) == 10:
        # Format: LL DD LL DDDD (Length 10)
        return (reg[0:2].isalpha() and
                reg[2:4].isdigit() and
                reg[4:6].isalpha() and
                reg[6:10].isdigit())
    return False

def format_receipt(reg):
    """Formats a valid registration number as 'TN 09 AB 1234'."""
    clean = normalised(reg)
    if is_valid(clean):
        if len(clean) == 9:
            return f"{clean[0:2]} {clean[2:4]} {clean[4:5]} {clean[5:9]}"
        else:
            return f"{clean[0:2]} {clean[2:4]} {clean[4:6]} {clean[6:10]}"
    return "INVALID"

if __name__ == "__main__":
    # Test cases including valid and invalid formats
    test_inputs = ["tn 09 ab 1234", "KA-01-M-0007", "TN9AB1234", "TN 22 1234 AB"]

    print("-" * 65)
    for raw in test_inputs:
        reg = normalised(raw)
```

```
    status = "valid" if is_valid(reg) else "INVALID"
    formatted = format_receipt(raw)
    print(f"{raw:15} -> {reg:11} {status} | Receipt: {formatted}")
print("-" * 65)
```



Q4 – Building Long Text: + or join()?

1. Performance Comparison Implementation

Benchmarks string construction across 200,000 lines comparing repeated += concatenation against a single str.join() call.

"""

Q4 – Building Long Text: + or join()?

Course: CSA0809 - Python Programming

Student: Y. Siva Pardhav (192412612)

```
"""

import time

# Generate 200,000 items as test parts
items = [f"Item{i}:Rs{i % 50}" for i in range(200_000)]

def build_plus(parts):
    """Builds string using repeated += concatenation inside a loop."""
    text = ""
    for p in parts:
        text += p + ";"
    return text

def build_join(parts):
    """TODO 1: Build the same text using a single str.join() call."""
    return ";".join(parts) + ";"

if __name__ == "__main__":
    print("--- Running String Building Performance Benchmark ---")

    # Benchmark repeated += operator
    start = time.perf_counter()
    a = build_plus(items)
    t_plus = time.perf_counter() - start
```

Benchmark single str.join() call

start = time.perf_counter()

b = build_join(items)

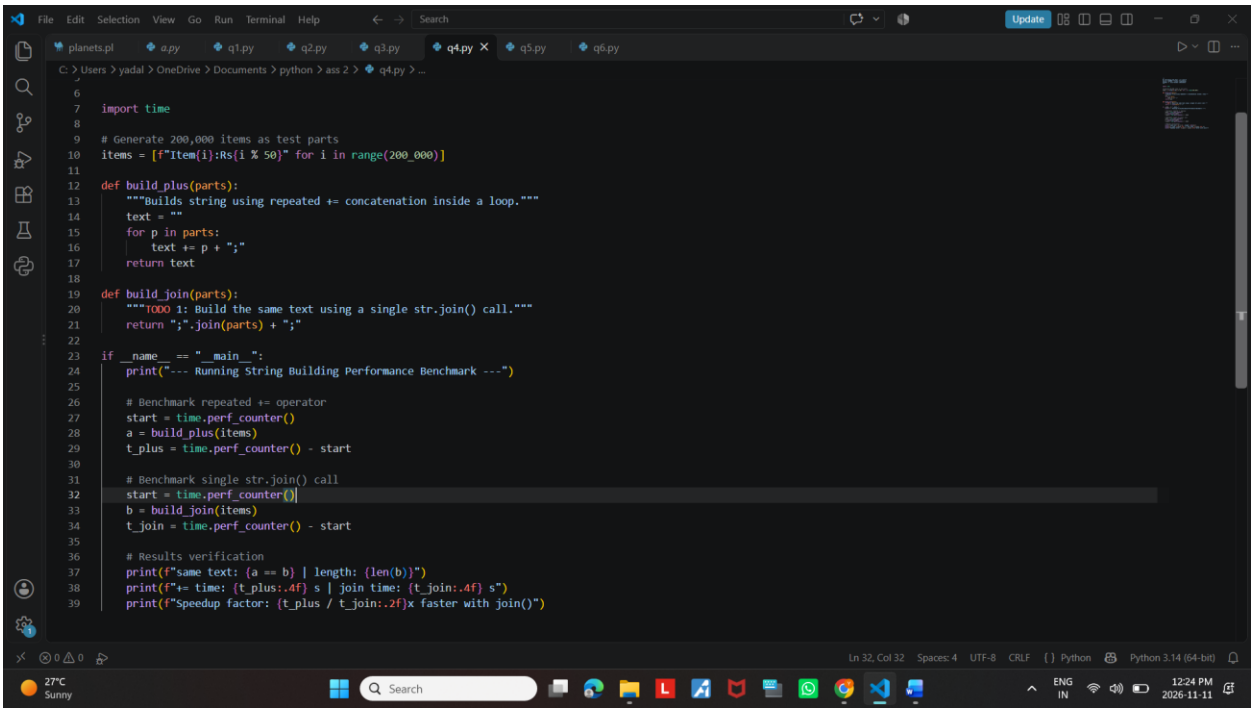
t_join = time.perf_counter() - start

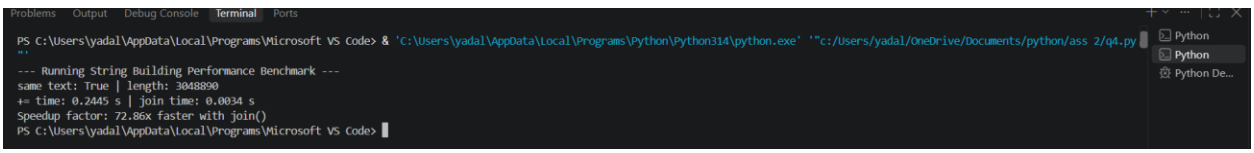
Results verification

print(f"same text: {a == b} | length: {len(b)}")

print(f"+= time: {t_plus:.4f} s | join time: {t_join:.4f} s")

print(f"Speedup factor: {t_plus / t_join:.2f}x faster with join()")





Q5 – Functions, Parameters and Scope

1. Fixing Default Parameter Mutation Bug

Default arguments in Python are evaluated once at function definition time. Using order=[] causes lists to persist across function calls. Setting order=None resolves this.

```
"""
Q5 – Functions, Parameters and Scope (College Canteen Order System)
Course: CSA0809 - Python Programming
Student: Y. Siva Pardhav (192412612)
"""


def add_item_buggy(item, order=[]):
    """
    Buggy version: Uses a mutable list as a default argument.
    The default list is created ONCE when the function is defined, not when called.
    """
    order.append(item)
    return order


def add_item(item, order=None):
    """
    Corrected version: Uses None as default and initializes a fresh list internally.
    """
    # TODO 1: Create a new empty list when order is None, append the item and return the list
    if order is None:
        order = []
    order.append(item)
```

```
    return order

# Demonstration of Scope (Local, Global, Nonlocal)
global_var = "I am Global"

def scope_demo():
    local_var = "I am Local"

    def inner_func():
        nonlocal local_var
        local_var = "Modified Local (Nonlocal)"

    inner_func()
    return local_var

if __name__ == "__main__":
    print("--- 1. Default Argument Bug Demonstration ---")
    print("Buggy calls (items persist from previous calls):")
    print("Call 1:", add_item_buggy("Tea"))
    print("Call 2:", add_item_buggy("Vada"))  # 'Tea' persists unexpectedly!

    print("\nCorrected calls (independent orders):")
    print("Customer 1:", add_item("Tea"))
    print("Customer 2:", add_item("Vada"))  # Starts fresh!
```

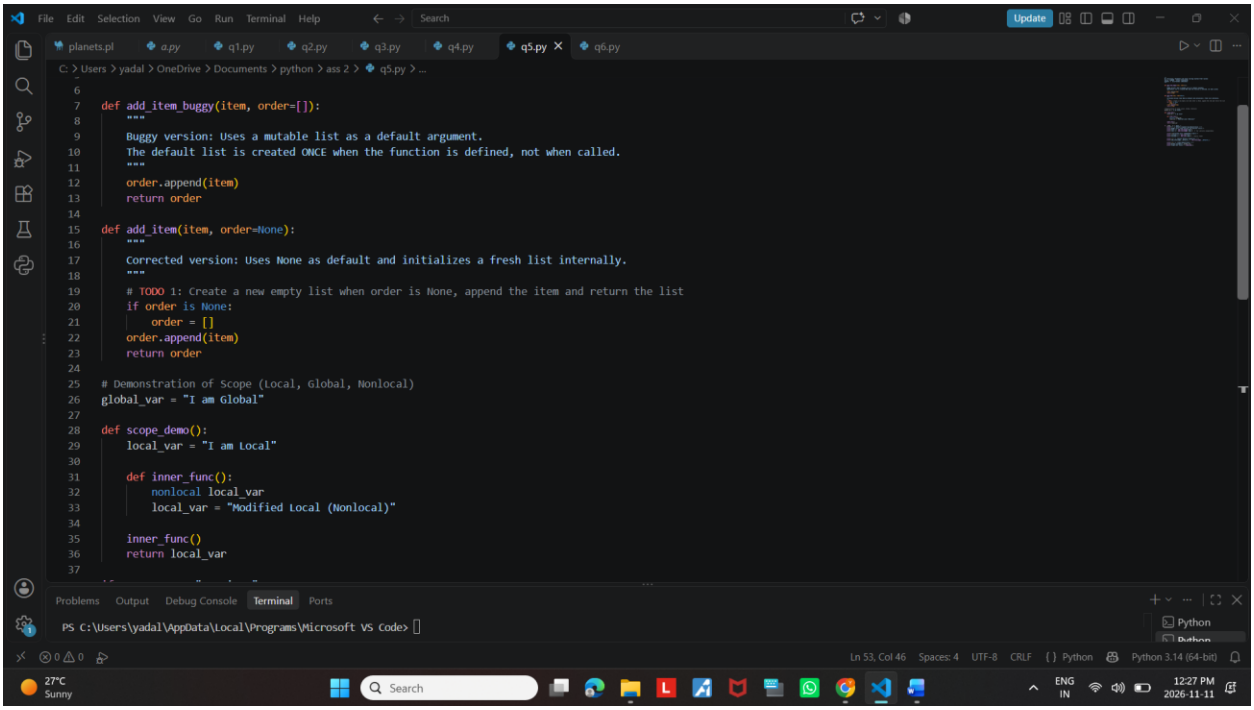
print("\n--- 2. Default Defaults Inspection ---")

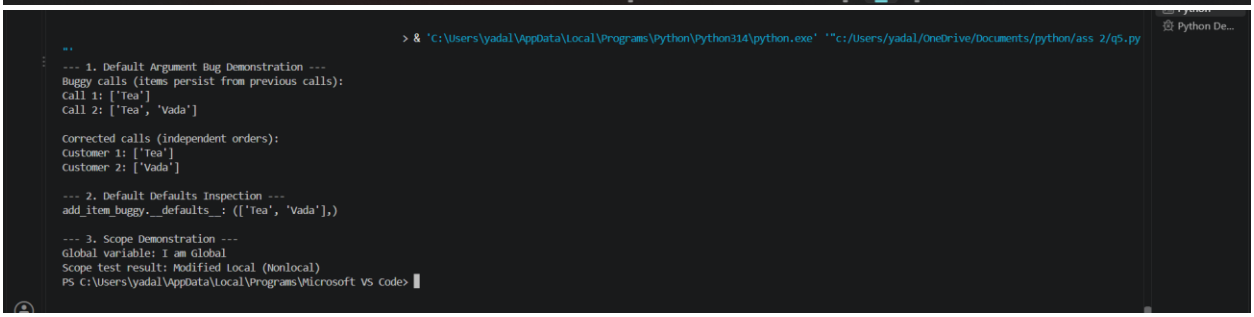
print("add_item_buggy.__defaults__:", add_item_buggy.__defaults__)

print("\n--- 3. Scope Demonstration ---")

print("Global variable:", global_var)

print("Scope test result:", scope_demo())





Recursion, Memoisation, and Iterative

1. Staircase Problem Implementation & Memoisation

Computing ways to climb n steps taking 1 or 2 steps at a time. Plain recursion yields O(2^n) exponential work. @lru_cache reduces this to O(n).

```
"""

Q6 & Q7 – Recursion, Memoisation, Limits, and Iterative Redesign

Course: CSA0809 - Python Programming

Student: Y. Siva Pardhav (192412612)

"""


from functools import lru_cache

import sys

import time


# Global counter for naive recursion call tracking

calls = 0


def ways_naive(n):

    """Plain recursive approach - O(2^n) time complexity with exponential call counts."""

    global calls

    calls += 1

    if n <= 1:

        return 1

    return ways_naive(n - 1) + ways_naive(n - 2)
```

```
@lru_cache(maxsize=None)
def ways_memo(n):
    """Memoised recursive approach - O(n) time complexity using lru_cache."""
    global calls
    calls += 1
    if n <= 1:
        return 1
    return ways_memo(n - 1) + ways_memo(n - 2)


def ways_iterative(n):
    """
    Q7: Optimized Iterative Bottom-Up Redesign.
    Achieves O(n) time complexity and O(1) auxiliary space complexity.
    Eliminates stack overflow and RecursionError for large inputs like n = 1000.
    """
    if n <= 1:
        return 1
    prev, curr = 1, 1
    for _ in range(2, n + 1):
        prev, curr = curr, prev + curr
    return curr


if __name__ == "__main__":
    print("--- 1. Q6 Call Count Comparison (n = 25) ---")
```

```
calls = 0
res_naive = ways_naive(25)
print(f"naive: {res_naive} | calls = {calls}")

calls = 0
res_memo = ways_memo(25)
print(f"memo : {res_memo} | calls = {calls}")

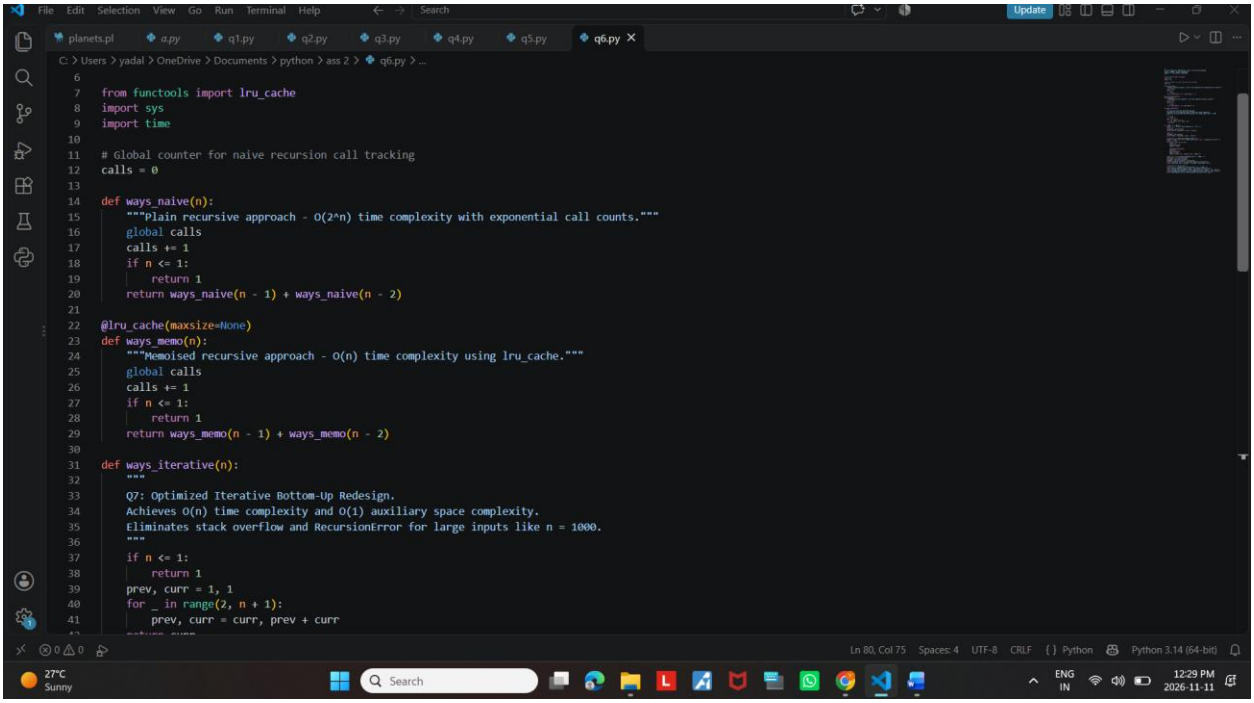
print("\n--- 2. Growth Investigation Table ---")
print(f"{'Steps (n)':<10} | {'Naive Calls O(2^n)':<20} | {'Memoised Calls O(n)'}")
print("-" * 55)
for n_val in [10, 20, 25, 30]:
    calls = 0
    ways_naive(n_val)
    naive_c = calls

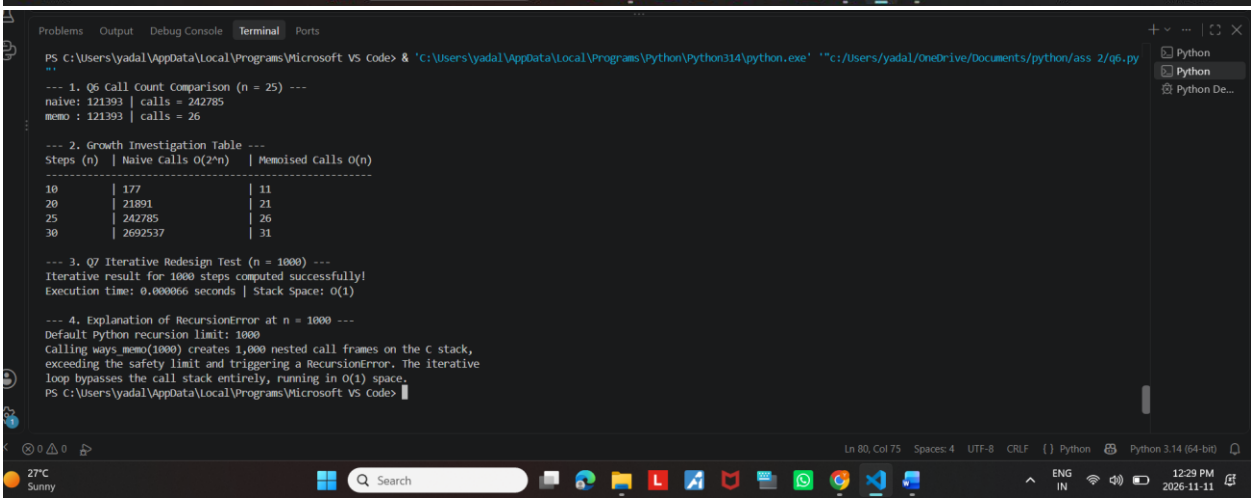
    ways_memo.cache_clear()
    calls = 0
    ways_memo(n_val)
    memo_c = calls
    print(f"{n_val:<10} | {naive_c:<20} | {memo_c}")

print("\n--- 3. Q7 Iterative Redesign Test (n = 1000) ---")
start_time = time.perf_counter()
ans_1000 = ways_iterative(1000)
```

```
elapsed = time.perf_counter() - start_time
print(f"Iterative result for 1000 steps computed successfully!")
print(f"Execution time: {elapsed:.6f} seconds | Stack Space: O(1)")

# Explanation of RecursionError
print("\n--- 4. Explanation of RecursionError at n = 1000 ---")
print(f"Default Python recursion limit: {sys.getrecursionlimit()}")
print("Calling ways_memo(1000) creates 1,000 nested call frames on the C stack,")
print("exceeding the safety limit and triggering a RecursionError. The iterative")
print("loop bypasses the call stack entirely, running in O(1) space.")
```


File Edit Selection View Go Run Terminal Help
Update
planets.pl a.py q1.py q2.py q3.py q4.py q5.py q6.py
C: > Users > yadal > OneDrive > Documents > python > ass 2 > q6.py > ...
from functools import lru_cache
import sys
import time
Global counter for naive recursion call tracking
calls = 0
def ways_naive(n):
"""Plain recursive approach - O(2^n) time complexity with exponential call counts."""
global calls
calls += 1
if n <= 1:
return 1
return ways_naive(n - 1) + ways_naive(n - 2)
@lru_cache(maxsize=None)
def ways_memo(n):
"""Memoised recursive approach - O(n) time complexity using lru_cache."""
global calls
calls += 1
if n <= 1:
return 1
return ways_memo(n - 1) + ways_memo(n - 2)
def ways_iterative(n):
"""
Q7: Optimized Iterative Bottom-Up Redesign.
Achieves O(n) time complexity and O(1) auxiliary space complexity.
Eliminates stack overflow and RecursionError for large inputs like n = 1000.
"""
if n <= 1:
return 1
prev, curr = 1, 1
for _ in range(2, n + 1):
prev, curr = curr, prev + curr
Ln 80, Col 75 Spaces: 4 UTF-8 CRLF { } Python Python 3.14 (64-bit)
27°C Sunny
Search
ENG IN
12:29 PM 2026-11-11


Problems Output Debug Console Terminal Ports
PS C:\Users\yadal\AppData\Local\Programs\Microsoft VS Code> & 'C:\Users\yadal\AppData\Local\Programs\Python\Python314\python.exe' 'c:/Users/yadal/OneDrive/Documents/python/ass 2/q6.py
''
--- 1. Q6 Call Count Comparison (n = 25) ---
naive: 121393 | calls = 242785
memo : 121393 | calls = 26
--- 2. Growth Investigation Table ---
Steps (n) | Naive Calls O(2^n) | Memoised Calls O(n)
10 | 177 | 11
20 | 21891 | 21
25 | 242785 | 26
30 | 2692537 | 31
--- 3. Q7 Iterative Redesign Test (n = 1000) ---
Iterative result for 1000 steps computed successfully!
Execution time: 0.000066 seconds | Stack Space: O(1)
--- 4. Explanation of RecursionError at n = 1000 ---
Default Python recursion limit: 1000
Calling ways_memo(1000) creates 1,000 nested call frames on the C stack,
exceeding the safety limit and triggering a RecursionError. The iterative
loop bypasses the call stack entirely, running in O(1) space.
PS C:\Users\yadal\AppData\Local\Programs\Microsoft VS Code>
Python
Python
Python De...
Ln 80, Col 75 Spaces: 4 UTF-8 CRLF { } Python Python 3.14 (64-bit)
27°C Sunny
Search
ENG IN
12:29 PM 2026-11-11